On-device intrusion detection on ESP32

Can a small device detect attacks on itself, and which models fit?

What is an IDS?

An IDS watches a device or a network for signs of an attack and raises an alert. It flags the attack, it does not stop it by itself.

- **Reads some data:** network traffic, or the device's own behaviour.
- **Makes a decision:** normal or attack
- **By data source:** network-based (gateway) or host-based (device).
- **By method:** signature / supervised (known attacks only) or anomaly (can catch unknown attacks).

Where does an IDS run?

Network-based usually runs on a **gateway** and sees all the traffic. Host-based runs on the **device** itself. For IoT today, the gateway is the common place; on the device is rare and hard.

A gateway or cloud IDS

- Good: sees the whole network, lots of memory and power, one place to manage.
- But: latency to reach it, needs connectivity, sees private household traffic, single point of failure.

So why not just run it on the device?

- **Tiny budget:** an ESP32 has a few hundred KB of memory and runs on battery.
- **No operating system:** host-based IDS (e.g. HADES-IoT) needs Linux system calls — a bare MCU has none. A standard, generic IDS is hard, because every MCU is different.
- **Limited view:** the device sees only its own traffic, not the whole network.
- **Real-time, single core:** it must keep up live and share the chip with the device's real job.
- **Dataset availability:** little data that matches what a device actually sees.

How host-based tools work

On normal computers, a small **agent** runs on each device and watches it (files, logins, running programs). It sends what it sees to a **manager** on a server, which decides if it is an attack.

Agent on the device → Manager on a server → Alert

Examples: OSSEC, Wazuh. The catch: they need a real operating system (Linux or Windows). A small IoT chip has none, so these tools can’t run on it.

People did try it for IoT

- **Still needs Linux — HADES-IoT (2022):** a small agent on Linux IoT devices. It watches which programs start and blocks unknown ones. Works well, but the device must run Linux.
- **Another way, no OS — Diwall (2023):** instead of system calls, it reads the chip's hardware counters. No OS needed, but it adds special hardware.

HADES-IoT: Breitenbacher et al., 2022. Diwall: El-Bouazzati, PhD thesis, 2023.

The data does not match the device

Public datasets were recorded in the middle of the network. They are captured at one central point that sees every device, so they are rich. A single device sees less.

- Dataset (central view): packet flags, connection counts, sizes, rates — everyone's traffic.
- Device (its own view): its own messages and connections, and some packet details depending on the firmware.

So a model trained on gateway data may not run on the device. We must do feature selection — pick features the device can really collect.

What the research did on MCUs

Paper	Hardware	Type	Dataset	Main limit
Javed 2024	ESP32	supervised trees	own — ESP32 traffic, on Kaggle	2 attacks only
Boi 2025	ESP32	supervised NN	public — NSL-KDD (non-IoT)	wrong features
Ieropoulos 2026	ESP32-S3	supervised, energy measured	own — 24 h on the ESP32	one network
Ortiz-Garcés 2026	Cortex-M4	NN (NAS)	public — IoT-23, Edge-IIoTset	shaky numbers

All **supervised** (known attacks only). Most built their **own small dataset**, because public ones don’t match what a device sees.

One model for all, or one per device?

Small models **do** fit in memory. That is not the main problem.

- **The data:** it must use features the device can really collect. Gateway datasets often don’t.
- **The transfer:** a model trained on one device may not fit another. A thermostat and a camera behave differently.

So the hard part is the data and whether it carries over, not the memory.

The gap

Very few detectors run on a microcontroller — most need Linux. And it is still unclear whether a device's own local features are enough to detect attacks.

Our focus

- IP-based Wi-Fi devices, using only the traffic one device sends and receives.
- Attacks aimed at the device (floods, scans) — not attacks between other devices.
- A dual-core ESP32, so detection does not block the device's real job.
- Benchmark several models; measure accuracy, energy and real-time response.
- Data: public datasets (with feature selection) and/or data collected from the device.
- Out of scope: BLE / Zigbee (non-IP), and whole-home correlation.

Possible direction: detect attacks on the device and benchmark models — cost, energy, detection rate, latency and real-time behaviour.